La programmation orientée objet

- La POO est un paradigme de programmation qui permet:
  - représenter les éléments d'un système sous forme d'objets
  - regrouper les données et les comportements
  - organiser le programme de manière modulaire, naturellement
  - une meilleure organisation du code
  - une abstraction du monde réel
  - une réduction de la complexité
  - une meilleure maintenabilité
  - une réutilisation du code

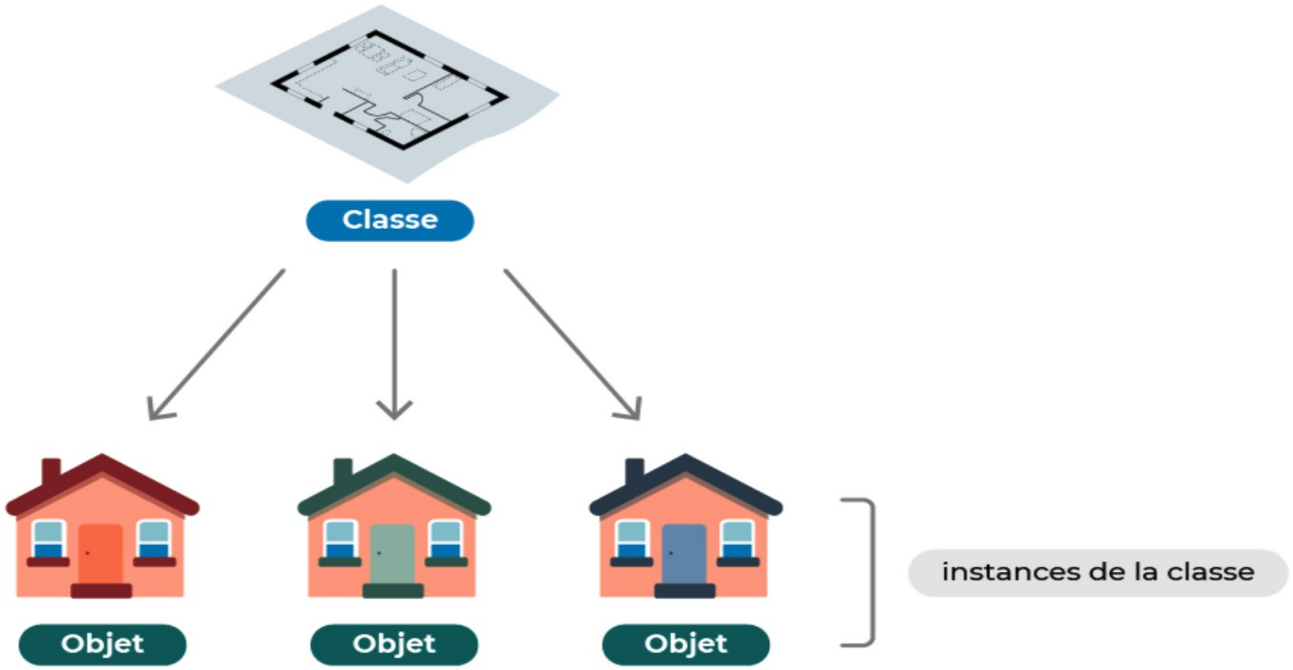
Classes et Objets

- Programmation orienté objet : POO
- Classe : généralisation de la notion de structure et de type
- Une classe contient:
  - Des données (données membres)
  - Des fonctions (fonctions membres ou méthode)
- Principe fondamental de la POO : L'encapsulation – les données ne sont accessibles que via des fonctions membres

Classe et objet

- La classe contient les spécifications d'un objet donné

- Un objet doit respecter toutes les spécifications qui se trouvent dans la classe correspondante



cin et cout à la place de printf, scanf

- Dorénavant, on utilisera cout à la place de printf
  - `printf("%d",x)` ⇒ `cout << x ;`
  - `#include <stdio.h>` ⇒ `#include <iostream>`

- Dorénavant, on utilisera cin à la place de scanf
  - `printf("%d",x)` ⇒ `cin >> x ;`
  - `#include <stdio.h>` ⇒ `#include <iostream>`

- `cout << x << y << "    "<< s <<endl ;`
- `cin >> x >> y >> y ;`

Rappel structure

```
struct point
  {
        double x;
        double y;
  } ;
point a, b; // permet de déclarer et de
            //créer deux variables a et b de
            //type point.
a.x = 3;
a.y = 2; //on accède aux données (ou champs
         //de la structure via « . »
```

Classe point : le fichier point.h

```
class point {
  private :
     double x;
     double y;

  public :
     void initialise(double, double);
     void deplace(double, double);
     void affiche();
  };
```

Classe point : fichier point.cpp

```
#include ''point.h''
void point::initialise(double abs, double ord)
{
   x = abs; y = ord;
}
void point::deplace(double  dx, double dy)
{
  x += dx ; y += dy;
}
void point::affiche()
{
  cout << ''(''<<x<<'',''<<y<<'')''<<endl;
}
```

Utilisation de la classe

```
#include ''point.h''

int main()
{
   point a, b;
   a.initialise (5.7, 2.3);
   a.affiche();
   a.deplace(3.5; 7.2);
   a.affiche();
   b.affiche();
}
```

Remarques

- Dans le jargon de la POO, on dit que a et b sont:
  - → des instances de la classe point.
  - → des objets de type point
- par défaut, x et y sont des données membres privées :
  - → En dehors des fonctions membres, l'instruction a.x = 5 est interdite
  - → Il faut prévoir des fonctions d'accès en lecture et en écriture
- Les fonctions membres sont ici publiques. Il est possible d'en avoir des privées

Remarques

- Les mots clés *public* et *private* peuvent apparaître à plusieurs reprises.
- Par défaut les membres d'une classe sont privés

```
class x
{
  private :
    .....
  public :
    ......
  private :
    .....
} ;
```

Exemple

```
class Personne
{
    string nom;
    int age;

    public :
    void sePresenter()
    {
        cout << "Je m'appelle " << nom << " et j'ai " << age << " ans.";
    }
};
int main()
{
   Personne P ;
   P.nom="Julien" ;
   P.age=25 ;
   P.sePresenter() ;
}
```

Compte bancaire

```
class CompteBancaire
{
        float solde;

        public :
        void deposer(float montant)
        {
                solde += montant;
        }

        void retirer(float montant)
        {
                solde -= montant;
        }
};
```

Etudiant

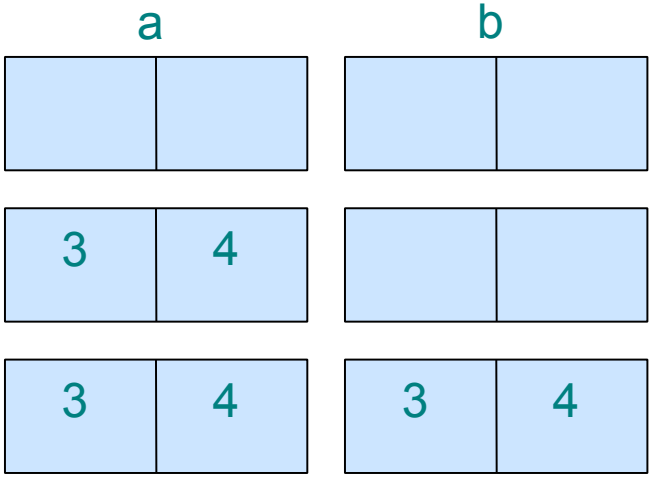
```
class Etudiant
 {
    string nom;
    float note;

    public :
    string mention()
    {
        if (note >= 16) return "Très bien";
        else if (note >= 14) return "Bien";
        else return "Passable";
    }
};
```

```
class Date
{
    int jour;
    int mois;
    int annee;
    public :
    void afficher()
    {
        cout << jour << "/"<<
        mois << "/"<< annee;
    }
};
```

```
class ProduitAlimentaire {
public:
    string nom;
    float prix;
    Date dateExpiration;

    void afficher() {
        cout <<"Produit :"<<nom
        <<endl;
        cout <<"Prix :"<<prix<<"Euros"
        << endl;
        cout << "Date d'expiration:";
        dateExpiration.afficher();
        cout << endl;
    }
};
```

Affectation d'objets

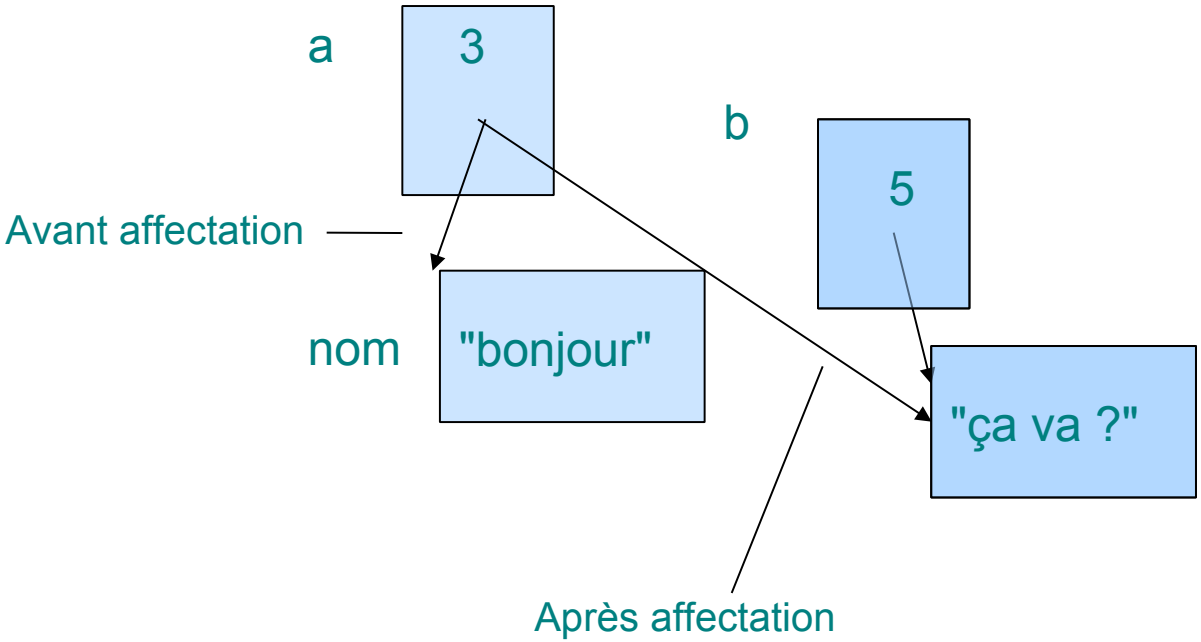
```
point a, b;

a.initialise(3.0, 4.0);
b = a;
```



- On ne peut pas écrire :
  - `b.x = a.x ; b.y = a.x,`
  - car x et y sont privées
- Notez que l'opération b=a est toujours légale. Elle marche lorsque les données ne sont pas allouées dynamiquement.

Attention à l'utilisation de « = »

```
class cla
{
   int x;
   char *nom;
}
cla a, b;
a=b;
```



Constructeur

- C'est une fonction appelée automatiquement à la création d'un objet.
- Son rôle est d'initialiser l'objet convenablement
- De faire des allocations dynamiques si nécessaire
- Il est toujours nécessaire d'en faire un ou plusieurs
- Le constructeur a le même nom que la classe et n'a pas de valeur de retour

Constructeur

- La rôle principal du constructeur est de garantir la cohérence des objets
- Un objet carré ne doit pas avoir une valeur négative comme longueur de l'un de ses côtés
- Un objet cercle ne doit pas avoir un rayon négatif
- Le pourcentage des intérêts pour un crédit doit être entre 0 et 100.
- .....

Exemple : la classe point

```
class point
{
  double x;
  double y;
  public :
  point (double , double)
  void deplacer (double , double);
  void afficher();
};
```

Exemple la classe point

- On ne peut plus faire cela:

```
point a;
```

- En fait, à partir du moment où un constructeur est défini, il doit être appelé lors de la création de l'objet.

```
point a(3.5, 6.2);
point b(0.0 , 0.0);
```

- C'est un bon moyen de garder la cohérence des objets

point.h

```
class point {
   private :
      double x;
      double y;
   public :
      point (double, double);
      void deplace(double, double);
      void affiche();
};
```

point.cpp

```
include ''point.h''

point::point(double abs, double ord)
{
  x = abs; y = ord;
}
void point::deplace(double  dx, double dy)
{
  x += dx ; y += dy;
}
void point::affiche()
{
  cout << ''(''<<x<<'',''<<y<<'')''<<endl;
}
```

Le main

```
#include ''point.h''

int main()
{
   Point a(5.0, 7.1);
   a.affiche();
   a.deplace(-1.0, 8.3);
   a.affiche();
}
```

Le destructeur

- Le destructeur est une fonction appelée au moment ou un objet doit disparaître
- L'appel au destructeur est fait automatiquement

```
......
{
  point  a(3.5, 5.5);
  ....
}
```

C'est ici que le destructeur est appelé

- Le destructeur a le même nom que la classe précédé d'un "~" : ~point()
- Le destructeur n'a pas d'arguments

La classe test

```
#include <iostream.h>
Classe test
{
    pulic :
      int num;
      test(int);
      ~test();
}
test::test(int n)
{
    num=n;
    Cout<<'' ++ appel du constructeur – num = '' << num<<endl;
}
test::~test()
{
    Cout<< ''- -  appel  du destructeur – num = '' <<  num <<endl;
}
```

La classe test

```
Void fct(int p)
{
  test x(2*p);
}
int main()
{
    test a(1);
    for(int i=1; i<=2; i++) fct(i);
}
```

```
-----------------------------------------------------------------
   --------
++ appel du constructeur – num = 1
++ appel du constructeur – num = 2
- -  appel  du destructeur – num = 2
++ appel du constructeur – num = 4
- -  appel  du destructeur – num = 4
- -  appel  du destructeur – num = 1
```

Les fonctions membre constantes

- `const int n=3;`
- `n=12;` est une instruction interdite
- En C++ une fonction peut être déclarer comme constante, ca veut dire:
  - la fonction ne peut pas modifier les données membres de la classe.

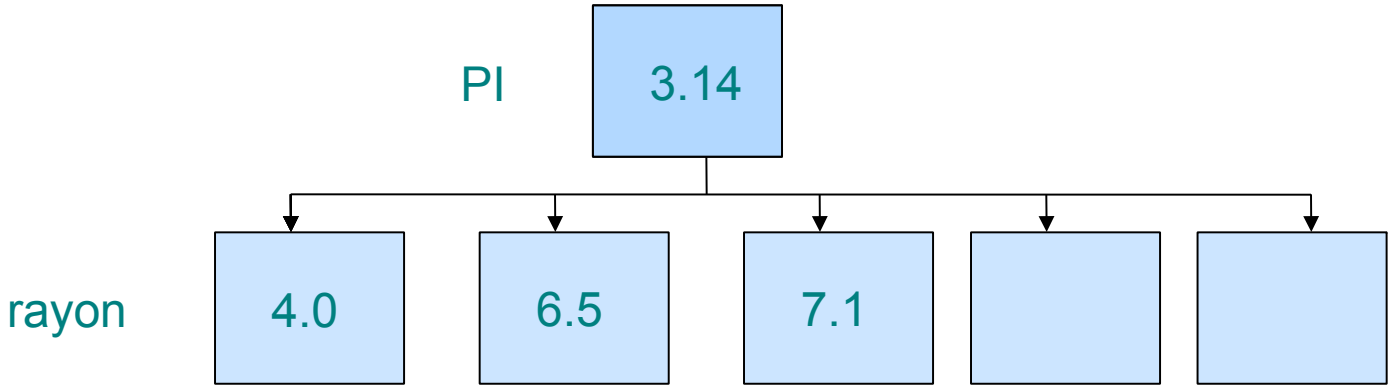
```
class point
{
    point (double x, double y);
    void affiche() const;
    void deplace(double dx, double dy);
}
```

Les données membre statiques

```
class cercle
{
    static double pi;
    double rayon;
    public :
    ......
    .......
}
```

Données membre statiques

- Tous les objets de type cercle partage la même donnée membre pi
- Économie de l'espace mémoire
- Cohérence



- Pour initialiser : double cercle::pi = 3.14;

Le nombre d'occurrences des objets

```
#include <iostream.h>
class cpt_obj;
{
        static int ctr;
        public :
                cpt_obje ();
                ~cpt_obj ();
}
int cpt_obj::ctr = 0;
cpt_obj::cpt_obj()
{
      ctr++;
      cout<<''++ il y a ''
   <<ctr<<endl;
}
```

```
cpt_obj::~cpt_obj()
{
ctr++;
Cout<<''- - il y a ''
   <<ctr<<endl;
}
void fct()
{
        cpt_obj u, v;
}
int main()
{
        cpt_obj a;
        fct();
        cpt_obj b;
}
```

Valeur par défaut

```
void initialise(double a=0.0, double b=0.0)
{
    x=a;
    y=b;
}
Que fait :
    point p(3.0,4.0);
    .....
    p.initialise();
    ..…
    p.initialise(3) ;
```

Les paramètres avec valeurs par défaut doivent être à droite.

```
void initialise(double a, double b=0.0)
```

Sur-définition de fonction

- On peut définir plusieurs fonctions avec le même nom.
- Pour les différencier, il faut que les arguments soient différents en nombre ou en types

```
void initialise(double a, double b)
void initialise (point & p);
Void initialise ( point *p);
```

Classe ensemble de points

```
class ens_points{
   point *tab;
   int nbe;
   int capacite
   public :
     ens_points(int cap);
     ~ens_points();
     bool present (point & p);
     affiche();
     bool inserer(point &p);
     int supprimer(point & p);
   }
```